

Course Information:

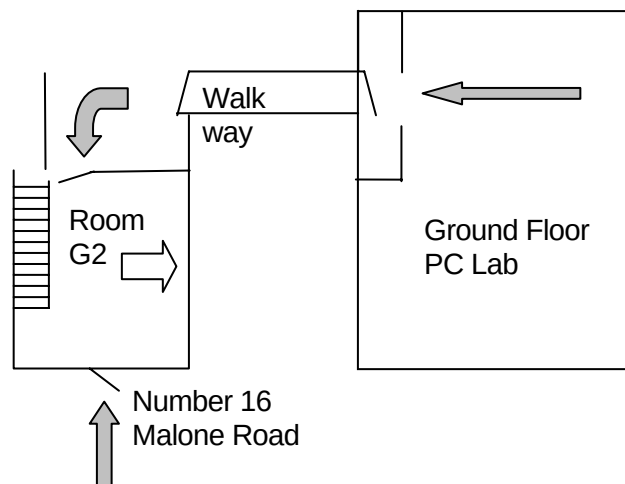
Course location and time	Crossland 1.15, Wednesday 9am-11am
Teaching Approach:	10 2-hour lectures over a 10-12 week period. Formative practical packs Open tutorial sessions Linkage to external resources for private study.
Lecture structure	1 st half: 9:05am-9:55am 2 nd half: 10:05am-10:55am
Recommended Textbook	H.M.Deitel & P.J. Deitel: <i>Java How to Program (5th Edition)</i> Prentice-Hall, 0-13-120236-7
WWW page	Queen's On-Line and http://www.cs.qub.ac.uk/~P.Hanna

Note, full information on the learning outcomes and their assessment can be found within the CSC735 Learning Outcomes and CSC735 Assessment documents.

Contact Details:

Contact details are as follows:

Philip Hanna	
Telephone:	x 4634 – (028) 90-274634
E-mail:	P.Hanna@qub.ac.uk
Room:	G2, 16 Malone Road (see map)
Times:	Pop over anytime



Going on the feedback provided from past years it appears that most problems can be readily answered through the use of E-mail, however, do feel free to pop into my office any time you wish.

Lecture plan:

The lecture plan followed within the course is as follows:

- Lecture 1: Programming / Debugging Strategies
- Lecture 2: Object Oriented Programming I
- Lecture 3: Object Oriented Programming II
- Lecture 4: GUI I: Events
- Lecture 5: GUI II: Components
- Lecture 6: GUI III: Containers
- Lecture 7: Multitasking / Threads
- Lecture 8: Exception Handling
- Lecture 9: File IO
- Lecture 10: Revision / Review (optional)

The CSC735 Learning Outcomes document provides full details of what you should get out of each lecture.

Course WWW page

URL: <http://www.cs.qub.ac.uk/~P.Hanna/>

Contents:

- Course information
- Lecture details/download (MS Word, PDF)
- Practical details (unassessed/assessed)
- Java On-line Resources

You should frequently visit the course WWW page. In particular, you will be able to find details of the weekly practicals.



The Java On-line resource section contains links to a number of excellent Java sites (offering an extensive range of tutorials, reference texts, advice, etc.). Hopefully, you will take a bit of time to explore some of the Java sites and get into the habit of using on-line resources. The reason I mention this is due to the considerable size of the Java language (in particular, the multitude of classes), thereby entailing that we can only explore a small subset of Java's capabilities within the course. Through the use of the linked resources, details on virtually any aspect of Java can be gathered.

Contents of lecture 1:

1. Skills of the software engineer

- Benefits of becoming a good programmer
- Skills of the software engineer
- The road towards becoming a good programmer

2. Debugging Strategies

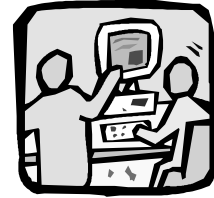
- The need for debugging skills
- Common compilation/run time errors

Skills of the Software Engineer

Benefits of becoming a good programmer

Firstly, let us answer the question of why we would want to become a good programmer:

- Make any MSc course/practical/examination involving Java more easily tackled (this basically encompasses every MSc course!).
- Greatly help with the summer project (the MSc project typically involves the development of a substantial Java program, certainly much larger than those encountered during the course practicals, because of this many students remark that they wish they had spent more time programming prior to the project).
- Improved job prospects. Obviously enough through improved examination marks, but also through the added confidence that arises from being familiar with a programming language.



The above reasons are somewhat particular to the needs and aims of MSc students. They are also fairly evident; however, the real challenge lies not in a recognition or acceptance of the above points, but in actually spending time (your free time) playing about with Java programs and thereby improving your programming skills (more on this later).

Skills of the software engineer

In this section I have tried to outline those skills that good software engineers (i.e. programmers) should possess. Given this, we can then ask how one might go about fostering and improving such skills.

Number 1: Clear/Precise Problem solving

Primarily software engineering is about problem solving. A programmer is given a problem description, for which they must devise a software solution, e.g. designing a telephone messaging service, debugging a program, expressing an algorithm in code, etc.



Evidently, to do this they need to initially understand the problem. The programmer must then be able to devise a solution to the problem. Failure to understand the problem or an inability to devise a software solution will obviously not result in a working software solution.

It is important to note that the type of problem solving employed by a programmer really only extends to the development of a clear solution expressed in terms of a number of small, successive steps (i.e. a solution that can be readily expressed in, for example, Java). In many respects how a programmer solves a software problem is similar to the process of building a Lego model or planning a complex car journey).

It might be worthwhile asking yourself how you would go about teaching someone with the aim of improving their problem solving skills. At least one way of accomplishing this would be to get the person to actually tackle and solve a lot of problems, thereby letting them automatically learn effective strategies and techniques. As a counterpart to this, you can improve your own problem solving skills in exactly the same manner.

Number 2: Finding out about / learning from

Computing is certainly one of the most rapidly changing work environments found today: software packages, operating systems and even programming languages have a limited 'shelf life' before they are replaced with something 'improved'. Additionally, it is very unlikely that any two software engineering projects will require exactly the same set of skills to solve. Because of this, software engineers are *expected* to be able to pick up any necessary computing skills/knowledge in order to successfully meet the demands of their current project.



Again, like problem solving this is a generic skill, i.e. the ability to learn and adapt. Nevertheless, given the ever-changing face of computing, it is a skill central to successful software engineering. As before, you might want to ask yourself how a programmer might go about trying to improve their ability to learn and adapt to new technologies.

Adaptability and Java

The ability to learn how to employ new classes is of particular importance given the huge set of libraries found within Java. The hundreds of classes and packages incorporated along with Java are designed to make the life of the programmer easier by providing readily reusable classes that offer commonly requested functionality. We could easily spend one entire module looking at nothing but GUI components. However, out of necessity only a thin slice of all available classes can be explored within one lecture.

Hopefully the provided links to on-line Java resources (available from the course web page) will make the task of finding out about new classes considerably more easy. Hence, being able to find out about and learn how to use new classes is a particularly useful skill for the Java programmer.

Number 3: Computing knowledge

Finally, we expect our good software engineer to have a wide range of computing experience, i.e. possessing knowledge of programming languages, design methodologies, algorithm design, quality management, team skills, etc.

It might appear strange that I have ranked problem solving skills and ability to learn as holding more importance than actual computing knowledge (however it reflects the fact that computer programming is all about problem solving in a rapidly changing field). This is not to say that computing knowledge is unimportant. Obviously it is important, and we should expect a good programmer to accumulate extensive computing knowledge as they gain experience.

It is worthwhile making a distinction between *generically applicable* knowledge and *implementation specific* knowledge. For example, within this course we will explore multitasking, looking at the basic concepts and also how it is implemented within Java. The concepts are transferable, meaning that they can be applied to any other language that supports multitasking, whilst the knowledge of how it is implemented within Java may not be transferable. Understandably, it is in the programmer's best interest to accumulate generically applicable knowledge whenever possible, as it is this knowledge they will be able to 'bring with them' to different projects.



The road towards becoming a good programmer

Given the above, how might we go about becoming a good programmer? Obviously, it is not sufficient to simply accumulate knowledge. Rather, it involves a process consisting of practice and the gradual formation and refinement of problem solving skills (as well as computing knowledge). This process is made considerably easier if the programmer is adaptable and able to hunt down and absorb new knowledge.

In many respects the approach is not dissimilar to the process of learning how to drive a car, as follows:

The Wrong Approach:



Take the Highway Code, various car manuals and other books on how to drive. Lock yourself in your room and intensively study. Attempt test. Fail test (i.e. we have accumulated knowledge but insufficient experience of applying that knowledge).

The Right Approach:



Study basics of driving, and then practice basics until confident. Build upon the basics by studying more complex material. Practice the newly learnt material until confident. Repeat. Attempt test. Pass test (i.e. we have accumulated knowledge and an ability to successfully apply that knowledge).

Likewise, in order to become good at programming it is not sufficient to solely study course material. Practical exploration (i.e. writing small programs, playing with the language) requires recall, understanding and application of learnt material – something that a simple presentation within a lecture is unlikely to ever achieve.

Coincidentally, apart from improving your grasp of Java, writing programs will also improve your problem solving abilities, learning skills, debugging skills, etc. – in fact, it really is a very effective means of improving programming skills.

Lectures and Practicals

All lectures on this course aim to firstly provide a theoretical introduction to a topic (e.g. OOP, GUIs, multithreading, etc.) and then follow this with details of how it is implemented within Java. In order to provide an opportunity to explore the material, each lecture will have an associated practical pack that will investigate the concepts covered within the lecture.

I certainly recommend that you extend the provided practicals (either to add additional functionality, try out some new graphical component, etc. – whatever tickles your fancy). Some of the on-line Java resources (linked from the course web page) might prove useful in this endeavor.

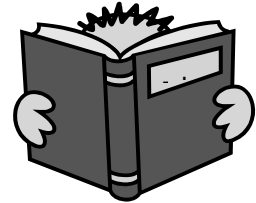
The Art of Debugging

Debugging Strategies

It is an inescapable fact that bugs, i.e. errors, will creep into your computer programs. The types of bug that may arise range from trivial typing mistakes to complex errors in logic, which may remain undetected for some time. Nevertheless, the net result is the same: the program does not correctly work.



Unsurprisingly, bugs should be hunted down and eradicated. Equally unsurprisingly, debugging programs is a task given to programmers. In fact, the testing/debugging phase of a project is likely to occupy a good percentage of the entire lifecycle (and hence a programmer's time). Given that testing/debugging programs is an integral part of software development, you might expect to find a healthy coverage of 'how to test/debug' in books, modules, etc.



Unfortunately, this is not really the case. Debugging is barely mentioned in most texts (note, it is possible to pick up the odd book on testing strategies, mostly as it forms an important part of the project lifecycle, however, such books explain how programs can be rigorously tested, not how bugs can be hunted down and eradicated).

There are two good reasons for the lack of debugging coverage:

1. The ability to debug is often viewed as a skill that the programmer automatically develops, i.e. something they 'picked up' when they were learning how to program.
2. How to debug a program is not something that can be clearly explained, apart from in terms of vague principles. In fact, becoming better at debugging is something that is learnt from actually correcting programs, not from reading books.

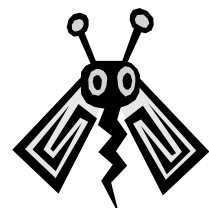
Hence, whilst it may not be possible to provide formulaic approaches guaranteed to remove program errors, it should be possible to provide some advice about how to attempt to find and remove bugs. This is my goal.

Aside: Programming kudos – the notion amongst programmers of who is a capable programmer, appears to be equally based on the ability to hunt down illusive bugs as it is on the ability to write complex pieces of code. It's a silly notion, but programmers will be programmers.

Types of program error

Programming errors comfortably fall into one of two different categories:

- **Compilation errors** – the program cannot be compiled, with various error messages issued.
- **Logical/run-time errors** – the program runs, but crashes or provides incorrect results, etc.



As a rule of thumb, compilation errors are easily identified, easily understood, and easily corrected (see aside). In contrast, logical/run-time errors are typically not as easily identified, understood nor corrected (mostly as they have more varied and complex reasons for existing).

Both types of error are now explored.

Caveat: The ease with which compilation errors can be removed is linked to the amount of experience the programmer has. For example, the first time a compilation error is encountered it is likely to require some time and effort to remove. By the 20th time that the compilation error has been encountered, it is likely to be easily removed (mostly as the programmer has had the experience of removing it 19 times before).

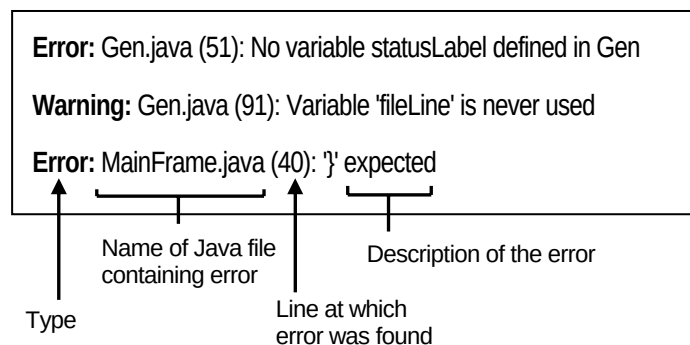
Because there are relatively few compilation errors, and they are common across all programs, it normally does not require too much time before a programmer can quickly remove this type of error.

Compilation errors

Compilation *errors* occur whenever the compiler cannot 'understand' the code, i.e. a syntax error has occurred. In such circumstances, the compiler provides the programmer with an error message, providing details of the type of error and its locality. Note, sometimes the compiler may issue warnings. Compilation *warnings* do not prevent the code from being compiled, and hence run, however, they should be considered as warning of potentially dangerous code that might result in a run-time error.

Removal of compilation errors

Shown below are examples of two compilation errors and one warning (taken from Symantec's VisualCafe):



The compiler's message indicates:

- If an error or a warning was generated
- The Java file in which the problem occurred
- The line at which the problem was found
- The nature of the problem

In all cases, the removal of compilation errors can be reduced to initially understanding the type of error (based on the message provided by the compiler). Using this, the error is then located within the code, and finally removed.

Note that, in certain circumstances, the removal of compilation errors can be a bit more involved, as follows:



More complex compilation errors

Two possibilities can complicate error detection and correction, as follows:

The line at which the compiler detects the problem may not be the line where the problem originated. For example, the **'}' expected** error means that the compiler was expecting to find an end of block, but didn't. In many cases, this is because the programmer forgot to close a block earlier in the program. Hence, it is important to remember that the compiler reports the line where it found the problem, which may be different than the line at which something needs to be fixed.

Knock on errors, i.e. one error causes a number of further errors in the program. For example, certain compilers will generate two errors, **';' expected** and **Identifier expected**, if given the following code snippet:

```
int iTest  
iTest = 14;
```

By correcting the first problem (adding **';'**), the second problem is removed (**iTest** is now defined). Hence, it is generally advisable to correct early errors first (i.e. those occurring at the start).

Common Java compilation errors/warning

Incorrect spelling of variables and keywords

Probably the most common problem reported by the compiler is incorrectly typed variable names or keywords. As Java is case sensitive this can easily occur, e.g. **iTest** and **itest**, etc.

The compiler may provide a variety of error messages. For example, a mistyped **while** loop, e.g. **whil(a==3)**, would generate a **';' expected** error as the compiler regards **whil()** as a method call. Typically misspellings in variable names, e.g. **iSpeltWront = 1;** will generate an **'Undefined variable: iSpeltWront'** message.



Return types from methods

Often a method will return the result of its computation via a return value, e.g. the method

```
int sum( int iFirstNumber, int iSecondNumber )
```

might be expected to return the summation of the arguments. When coding the **'sum'** method, if the programmer forgets to return a value when leaving the method then the following error will arise: **'Return required at end of int sum(int, int)'**. Alternatively, if an attempt is made to return a different type of value (e.g. a float instead of an int) then the corresponding error is: **'Incompatible type for return. Explicit cast needed to convert float to int'**.

Incorrect or insufficient method arguments

Consider the **'sum'** method defined above. The compiler will generate an error message should the programmer not pass two integer variables to this method (e.g. only one integer might be passed, or alternatively, a float may be passed). Typical messages are **'No method matching sum() found in MainFrame'** and also **'Incompatible type for method. Explicit cast needed to convert float to int'**

Incorrectly spelling a method name

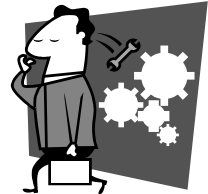
Assume that the `sum( int, int )` method defined above is part of a wider class, say, `Payroll`. Given the following code:

```
Payroll batchRun = new Payroll();
int iSum = batchRun.sun( 2, 6 );
```

then the compiler would generate the following error: **'Method sun(int,int) not found in Payroll'**

Exception handling

Often in Java a method will be called that may potentially throw an exception should problems arise (e.g. for opening a disk file, establishing a network connection, etc.). It is the programmer's responsibility to either catch and process the exception, or alternatively pass it on. Should an exception not be caught/thrown then the compiler will generate an error message as follows (assuming a disk file is being opened): **'Error: Exception java.io.IOException must be caught, or it must be declared in the throws clause of this method'**. Exception handling is more fully explored within lecture 8.



Problems with ; () { }

Another very common compilation problem encountered in languages such as C++ and Java involves either forgetting to place a semi-colon at the end of statements, or alternatively having a different number of left braces as compared to right braces (ditto with brackets, etc.). Consider the following code:

```
if( a == b )
{
    if{ a != c}
    {
        result = (a + c)*((c +d)*2 + e);
    }
}
```

If a semi-colon is omitted or the number of left and right brackets do not tally, then the compiler will respond with a **' ; Expected,) Expected**, etc. message. A word of caution, misplaced or forgotten braces, '{' and '}', can sometimes lead to obscure error messages as often the compiler cannot determine if a brace has been omitted until it has parsed the rest of the method, e.g.

```
public boolean checkParms( int x, int y )
{
    if( x < 0 || y < 0 )
    {
        System.out.println( "Invalid." ); return false;

    if( x < y )
        return false;
    else
        return true;
}
```

will be parsed by the compiler as follows:

```
public boolean checkParms( int x, int y )
{
    if( x < 0 || y < 0 )
    {
        System.out.println( "Invalid." ); return false;

        if( x < y )
            return false;
        else
            return true;
    }
    ?? '}' expected, not found ??
```

It is not until the compiler cannot find an expected brace that an error will be generated. Hence, in cases where an error cannot be readily found at the reported line, it is most probably due to a knock-on problem that occurred earlier.

Problems with different data types

Java is a 'strongly typed' language which means, in part, it is very strict when interpreting statements. A consequence of this can be seen when comparing different object types (i.e. when objects or data are compared to one another, e.g. is one String longer than another, etc.).

Java requires that the two items for comparison can safely be compared to one another. For example, in C/C++, Boolean true is represented as non-zero and false as zero, hence, it was valid to ask if an integer value was true or false, e.g.

```
int iThisValueIsTrueUnderC = 4;
if( iThisValueIsTrueUnderC == true ) { ... }
```

In Java the following error message will be generated: **'Incompatible type for if. Can't convert int to Boolean'**.

A similar error message will also be generated for comparisons of integers to floats, doubles to chars, etc. Hence, within Java, it is often necessary to cast (i.e. convert) one object type into another before the compiler will permit a comparison to be carried out.

Deprecation

Java has undergone a number of changes during its transition from version 1.0 to the present Java 2 range of platforms.

During this time some of the provided classes have changed (to reflect better design, additional features, etc.). What this means is that the later versions of Java support improved ways of accomplishing certain tasks (e.g. handling dates, processing events, etc.).

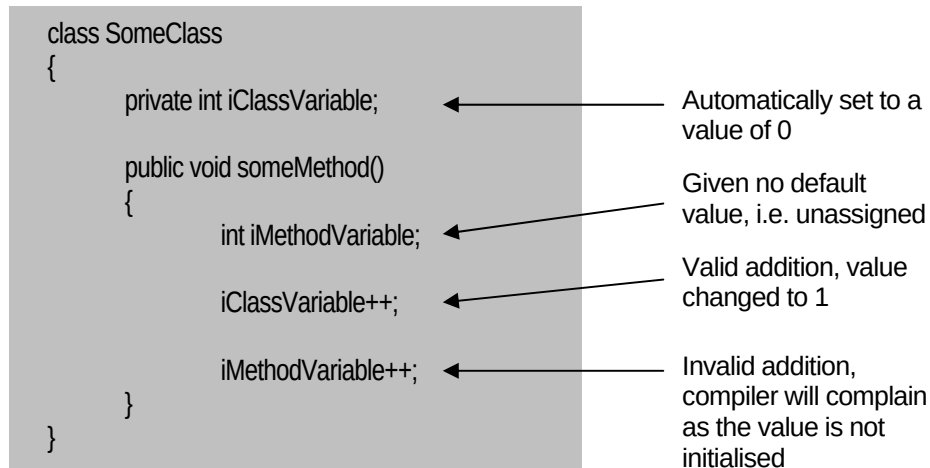
For reasons of compatibility with existing code, it is not practical to remove the 'old' way of doing things, instead the outdated methods are marked as 'deprecated', informing the programmer that the use of these methods is not recommended, and that an improved approach exists.



Programmers are well advised to stay away from using deprecated methods, unless the need for backwards compatibility is of paramount importance (this is only really likely to occur when writing an Applet that must run on as wide a range of browsers as possible).

Variables not initialised

In method calls, variables defined within the method are not initialised to default values (class variables, etc. are always initialised). This is for reasons of performance. In order to make sure the variable is correctly used, the compiler checks if it has been assigned a value before the variable is manipulated. If the compiler thinks a variable is being used without first being assigned a value (it can sometimes be wrong about this), then the following error message is generated: **'Variable iIndex may not have been initialised'**.



The above represents only a small selection of all possible compilation errors. Fortunately, most compilation errors are readily solved once the compiler's error message is understood. We now turn our attention to logical/run-time errors.

Debugging Logical/Run-time Problems

As mentioned, this type of program error often proves more problematic to solve. In part, this is simply due to there being a lot more possible run-time errors than there are compilation errors (i.e. each program will have a specific set of possible run-time errors that differs from other programs, whilst compilation errors tend to be common across all programs).

In what follows an iterative three-step procedure will be outlined, indicating a general technique that can be followed in order to track down and hopefully remove run-time errors. Before the three-step procedure is detailed we will first explore two examples of run-time problems and see how they can be resolved.

Example 1: Exception thrown

Suppose we have a program that accepts a String object made up of numbers, e.g. "12 2 43", and converts it into an array of Integer objects. The program operates on the assumption that the String will contain five integers. However, when the program is run an `ArrayOutOfBoundsException` is thrown. The code (no imports, comments) is as follows:

```
public class array
{
    static public void main(String args[])
    {
        StringTokenizer tokenizer = new StringTokenizer( "1 23 1 43 6" );

        Integer iData[] = new Integer[5];

        for( int iIndex = 1; iIndex <= 5; iIndex++ )
        {
            iData[iIndex] = new Integer( tokenizer.nextToken() );
            System.out.println( "\nNumber " + iIndex + " stored." );
        }
    }
}
```

The output from the above code fragment is:

```
Number 1 stored.
Number 2 stored.
Number 3 stored.
Number 4 stored.
java.lang.ArrayIndexOutOfBoundsException
Stack trace is disabled.
```

Given that an `ArrayIndexOutOfBoundsException` was thrown, we know at some point we have tried to access an invalid array position. Also, as the exception was not thrown until after the 4th value, we know the problem arose when processing the 5th number. From an understanding of what the program does, we can hypothesise that the crash is either due to the array being too small to accommodate five values, or alternatively, the 5th value is inserted at an invalid position.

Referring to the code, we can see that the problem is indeed due to an invalid array index (i.e. when `iIndex` takes on the value of 5).

Aside: What happens if we 'correct' the program by making the change: `for( int index = 0; index < 5; index++)`? Will it operate as expected?

The above problem was easily solved; however, what is noteworthy is the process:

1. Study the actual output from the program.
2. Based on an understanding of what was expected from the program produce a hypothesis as to the problem's cause.
3. Test the hypothesis.
4. Solve the problem, or repeat if necessary.



Example 2: Upping the complexity

Assume we have a program that performs a reasonably complex (and utterly useless) operation upon a String of numbers:

- Firstly, the program converts the String object into a corresponding array of floats.
- Next, the average of the numbers is determined.
- The overall result is obtained by multiplying the average by the smallest number in the array.

For example, an input string of "1 2 3" would give us an array [1],[2],[3], for which the average value is $(1+2+3)/3 = 2$, with the overall answer being $2 \text{ (average)} * 1 \text{ (smallest value)} = 2$. The code is as shown:

The program has been set up to use the String "5.0 10.0 5.0 15.0 15.0", which should give an expected result of 50 (Average value = $(5+10+5+15+15)/5 = 10$, smallest value = 5, result = $5*10$). However, when the code is run the following output is obtained:

```
Input = 5.0 10.0 5.0 15.0 15.0
Result = 0.0
```

In this case the program does not crash; however, it does not return the correct value. To complicate matters the output from the program does not really provide us with any information as to where the problem rests. How do we correct the problem in this case?

```
import java.util.*;
public class array
{
    static public void main(String args[])
    {
        String startingData = "5.0 10.0 5.0 15.0 15.0";

        float workingData[] = extractData( startingData );
        float average = determineAverage( workingData );
        float result = determineResult( average, workingData );

        System.out.println( "\nInput = " + startingData );
        System.out.println( "\nResult = " + result );
    }

    // Convert from String to float[]
    static private float[] extractData( String startingData )
    {
        StringTokenizer tok =
            new StringTokenizer( startingData );

        float data[] = new float[ tok.countTokens() ];
        for( int iIdx = 0; iIdx < data.length; iIdx++ )
            data[iIdx] = new Float( tok.nextToken() ).floatValue();

        return data;
    }
}
```

```
// Work out the average of float[]
static private float determineAverage(
    float[] workingData )
{
    float total = 0;
    for( int iIdx = 0; iIdx < workingData.length; iIdx++ )
        total += workingData[iIdx];

    return total / workingData.length;
}

// Multiply average by smallest value in float[]
static private float determineResult( float average,
    float[] workingData )
{
    int iSmallest = 0;
    for( int iIdx = 1; iIdx < workingData.length; iIdx++ )
        if( workingData[iSmallest] > workingData[iIdx] )
            iSmallest = iIdx;

    return average * iSmallest;
}
}
```

The answer lies in performing some detective work. We know the program can be separated into three different stages: conversion, average and result. We also know what the expected results are after each stage. Given this, if we print out the results after each stage we will be able to determine the region in which the problem arose (assuming the problem is localised to one particular area). The code to print out the results is as follows:

```
String startingData = "5.0 10.0 5.0 15.0 15.0";

float workingData[] = extractData( startingData );
for( int i = 0; i < workingData.length; i++ )
    System.out.println( "\nArray["+i+"] = "
        + workingData[i] );

float average = determineAverage( workingData );
System.out.println( "\nAverage=" + average );

float result = determineResult( average, workingData );

System.out.println( "\nInput = " + startingData );
System.out.println( "\nResult = " + result );
```

When run the results are as shown opposite. The numbers are correctly extracted and the average correctly determined, hence, thereby suggesting that the error lies within the last part of the program.

```
// Multiply average by smallest value in float[]
static private float determineResult( float average,
    float[] workingData )
{
    int iSmallest = 0;
    for( int idx = 1; idx < workingData.length; idx++ )
        if( workingData[iSmallest] > workingData[idx] )
            iSmallest = idx;

    return average * iSmallest;
}
```

```
Array[0] = 5.0
Array[1] = 10.0
Array[2] = 5.0
Array[3] = 15.0
Array[4] = 15.0
```

```
Average=10.0
```

```
Input = 5.0 10.0 5.0 15.0 15.0
Result = 0.0
```

Using a similar detective-like approach, the `determineResult` method can be tested to see if it is correctly finding the smallest index by adding the line **`System.out.println("\nSmallest = "+iSmallest);`** after the **for** loop. If this is done, a value of 0 is returned, indicating that the lowest value was correctly found.

If we look at the next line, **`return average * iSmallest`**, we are not returning the lowest value, but rather the index of the lowest value, i.e. 0, which entails that the overall result is zero. Hence we have found our error.

This example is more complex; in order to correct the problem it was necessary to break the program up into a number of sections, and then determine which section contained the error. The process of reduction then continued until the error was found.

The debugging process, illustrated in the last two examples, can be generalised into a three step procedure, as follows:

Step 1: Be aware of what the expected output should be for the given input. In other words, be aware of the program's expected behavior.

Step 2: Analyze the actual output from the program for clues as to the type and origin of the problem.

Step 3: Iterative Procedure: In essence this step involves making an informed decision, sometimes a guess, of where the problem arises within the program through an understanding of the expected and actual behavior of the program. Next, the program should be tested to see if the problem has been located.

The procedure is iterative as the process of selecting an area and then testing for the problem continues until the programmer has found the error.

Evidently, the above procedure is somewhat general in its formulation; however, this is necessary given the range of possible run-time problems.

Practical 1

After this lecture you should explore the first practical pack which should enable you to investigate the material in this lecture.

Learning Outcomes

Once you have explored and reflected upon the material presented within this lecture and the practical pack, you should:

- Appreciate that practical experience is a necessary component of becoming a competent programmer.
- Be able to identify the key skills required to be an effective software engineer.
- Be capable of removing all common compilation errors from a program.
- Understand how logical/run-time errors can be removed and be capable of removing simple logical/run-time program errors.

More comprehensive details can be found in the CSC735 Learning Outcomes document.

